USER MANUAL

FPGA-Accelerated LeNet MNIST Inference

Python Firmware Guide for the PYNQ Framework

Platform: PYNQ-ZU FPGA
Environment: Jupyter Notebook (PYNQ v2.x)
Model: LeNet-5 (INT8 Quantised) — MNIST Digit Recognition

Table of Contents

1. Introduction 3
2. System Overview 4
3. Prerequisites & Setup 6
4. Required Files 8
5. Code Walkthrough 9
   5.1 DriverConfig Class 9
   5.2 LeNetInt8Inference Class 16
   5.3 Main Execution Script 24
6. API Reference 27
   6.1 DriverConfig API 27
   6.2 LeNetInt8Inference API 32
7. Running Your First Inference 34
8. Understanding the Output 36
9. Troubleshooting 37

1. Introduction

This manual explains how to use the Python firmware that programs an FPGA-based hardware accelerator to run a LeNet-5 convolutional neural network for handwritten digit recognition (the MNIST task).

If you are reading this, you likely have access to:

- A pre-programmed FPGA board (the hardware accelerator is already built into the bitstream — you do not need to modify the hardware).
- A bitstream file (.bit) that configures the FPGA with the accelerator logic.
- A set of Python files that act as the "firmware" — the software that talks to the hardware.

Your only job is to run the Python code from a Jupyter Notebook on the PYNQ board. The FPGA accelerator is treated as a black box: you send it data, configure it, tell it to run, and read back results.

> **NOTE:** *You do not need to understand the internal hardware design of the FPGA. All hardware interaction is handled automatically by the Python driver described in this manual.*

What This System Does

The system takes a 28×28 pixel grayscale image of a handwritten digit (0–9) and predicts which digit it is. It does this by running a LeNet-5 neural network across two compute engines:

Compute Engine	What it handles
FPGA Hardware Accelerator	Convolution layers + Max-pooling layers (hardware-accelerated)
Python (Software)	Fully-connected (dense) layers, final classification

Who This Manual Is For

This manual is written for someone who:

- Is comfortable with basic Python programming.
- Has access to a PYNQ board with the correct bitstream and model files.
- Does not necessarily know anything about FPGAs or hardware design.
- Wants to understand how the firmware works and be able to run inference on their own images.

2. System Overview

High-Level Architecture

The diagram below summarises how all the pieces connect together. The FPGA board contains both a regular processor (ARM CPU) and reprogrammable logic (the FPGA fabric). The Python firmware runs on the CPU and controls the hardware accelerator in the FPGA fabric.

```
                    [ User: Jupyter Notebook on PYNQ board ]
                                  |  Python API calls
                                          v
              [ DriverConfig & LeNetInt8Inference (Python) ]
                        |  MMIO register writes / CDMA transfers
                                          v
                        [ AXI Interconnect on FPGA ]
                             /          |          \
                [ CDMA 0 ]  [ CDMA 1 ]  [ ControlSig IP ]
                 (image DMA) (kernel DMA) (start/status regs)
                              |                    |
                [ BRAMs ]           [ Conv/Pool Hardware Core ]
```

Key Hardware Components (Glossary)

The following table explains the hardware components that the Python firmware interacts with. You do not need to modify these — understanding their role is sufficient.

Component	What it does
`Overlay / Bitstream (.bit)`	The FPGA configuration file. Loading this file programs the hardware.
`BRAM (Block RAM)`	Fast on-chip memory inside the FPGA. Stores the image pixels and kernel (filter) weights during computation.
`CDMA (Central DMA)`	A DMA engine that moves data between main CPU memory (DDR) and the BRAMs automatically, without tying up the CPU.
`ControlSig IP`	A custom hardware block that exposes control registers. The firmware writes to these registers to start convolution, set image dimensions, enable ReLU, etc.
`MMIO`	Memory-Mapped I/O. A technique where hardware registers are accessed like regular memory addresses in Python.
`AXI Bus`	The internal communication bus connecting all hardware blocks on the FPGA.

Inference Flow Summary

When you call predict_digit(), the following sequence of operations happens automatically:

1. Image is loaded from disk, resized to 28×28, and quantised to INT8 format.
2. Convolution Layer 1 — image and kernel weights are sent to the FPGA BRAM via CDMA, the hardware runs the convolution, and results are read back.
3. Max-Pool Layer 1 — the output of Conv1 is passed back to the FPGA for 2×2 max-pooling.
4. Convolution Layer 2 — repeated for each of the 6 input channels, accumulating results.
5. Max-Pool Layer 2 — another 2×2 pooling step in hardware.
6. Fully-Connected Layers (FC1, FC2, FC3) — run entirely in Python/NumPy (software).
7. The digit with the highest score is returned as the predicted class (0–9).

3. Prerequisites & Setup

Hardware Requirements

- PYNQ-compatible board (e.g., PYNQ-ZU) with the LeNet accelerator bitstream loaded.
- The board must be powered on and accessible via its Jupyter Notebook interface (typically the board's IP address).

Software Requirements

The PYNQ board comes with these pre-installed. No manual installation is needed unless otherwise stated.

Package	Version	Purpose
`pynq`	2.x	Core FPGA overlay + MMIO + allocate
`numpy`	1.x+	Numerical arrays and matrix ops
`Pillow (PIL)`	Any	Image loading and resizing
`matplotlib`	Any	Displaying input images

Setting Up the Jupyter Notebook Environment

Step 1 — Connect to the board. Open a web browser and go to the PYNQ board's IP address (the default is shown on the board's LCD display).

Step 2 — Log in. The default password is xilinx.

Step 3 — Upload files. Using the Jupyter file browser, upload the following into the same folder:

- The Python firmware file (e.g., LeNet_cdma_try1.ipynb notebook)
- The bitstream file: LeNet_cdma_try4.bit

- The model weights file inside a subfolder: LeNet_data/lenet_int8_fpga_working_95.npz
- Your test image file (e.g., digit2.png)

Step 4 — Open a new Jupyter Notebook and either import the firmware module or paste the code into cells.

> **NOTE:** *All files must be in the same working directory as your notebook, unless you specify absolute paths.*

Quick Verification

Before running the full inference pipeline, confirm the PYNQ environment is working by running this in a notebook cell:

```
from pynq import Overlay
import numpy as np
print('PYNQ ready:', np.__version__)
```

You should see output similar to: PYNQ ready: 1.24.x — with no error messages.

4. Required Files

The table below lists every file required to run the system and explains its role.

File	Type	Description
`LeNet_cdma_try4.bit`	Bitstream	Programs the FPGA with the hardware accelerator logic. Must match the board exactly.
`LeNet_cdma_try4.hwh`	Hardware Handoff	Describes the hardware IP blocks to PYNQ. Placed in the same folder as the .bit file. Required by Overlay().
`LeNet_data/lenet_int8_fpga_working_95.npz`	Model Weights	Stores all INT8 quantised weights, biases, and quantisation parameters for the LeNet model.
`LeNet_cdma_try1.ipynb)`	Python Firmware	Contains DriverConfig and LeNetInt8Inference classes. This is the main code described in this manual.
`digit2.png (or any digit image)`	Test Image	A grayscale image of a handwritten digit. Must be convertible to 28x28 pixels.

IMPORTANT: *The .bit and .hwh files must have exactly the same filename (only the extension differs). PYNQ requires both files to be present in the same directory for Overlay() to work.*

Model Weights File — Contents

The .npz file contains the following NumPy arrays. These are loaded automatically by LeNetInt8Inference — you do not need to extract them manually.

Key in .npz file	Meaning
conv1_w, conv1_b	Weights and biases for Convolution Layer 1
conv2_w, conv2_b	Weights and biases for Convolution Layer 2
fc1_w, fc1_b	Weights and biases for Fully-Connected Layer 1
fc2_w, fc2_b	Weights and biases for Fully-Connected Layer 2
fc3_w, fc3_b	Weights and biases for Fully-Connected Layer 3 (output)
ScaleFactor, RoundFactor, Shift	Quantisation parameters used to rescale intermediate INT32 accumulations back to INT8

5. Code Walkthrough

This section walks through the firmware code step by step. The code is split into two main classes and a main script. Reading this section will give you a clear understanding of what each part does and why.

5.1 DriverConfig Class

This class is the low-level hardware driver. It handles everything related to talking to the FPGA — loading the bitstream, moving data in and out, and sending control signals. Think of it as the bridge between Python and the FPGA chip.

5.1.1 Imports and Class Definition

The firmware begins by importing all necessary libraries:

```
from pynq import Overlay, MMIO, allocate
import numpy as np
import time
from PIL import Image
import matplotlib.pyplot as plt
```

Import	Why it is needed
pynq.Overlay	Loads the FPGA bitstream and discovers all IP blocks in it
pynq.MMIO	Provides read/write access to hardware registers via memory addresses
pynq.allocate	Allocates physically contiguous memory buffers in DDR RAM (required for DMA)
numpy	Handles all array manipulation, data packing, and numerical operations

PIL.Image	Loads and resizes input images
time	Used to measure inference latency

5.1.2 Constructor: __init__()

When you create a DriverConfig object, the constructor sets up all the configuration variables but does NOT yet talk to the hardware. Hardware interaction begins only when you call load_overlay().

```
driver = DriverConfig(
    bitfile='LeNet_cdma_try4.bit',
    FMAP_IN_BRAM_ADDR  = [0xC0000000],
    KERNEL_BRAM_ADDR   = [0xC0000000],
    FMAP_OUT_BRAM_ADDR = [0xC2000000],
    lanes_per_bram=16,
    signed_bytes=True,
    depth_image_bram=2048,
    depth_kernel_bram=256,
)
```

Each parameter is explained below:

Parameter	Explanation
bitfile	Path to the FPGA bitstream (.bit file). Must be in the same directory as the notebook.
FMAP_IN_BRAM_ADDR	Physical memory address of the BRAM where input feature maps (image data) are written.
KERNEL_BRAM_ADDR	Physical memory address of the BRAM where kernel (filter) weights are written.
FMAP_OUT_BRAM_ADDR	Physical memory address of the BRAM from which output feature maps are read after computation.
lanes_per_bram	Number of parallel data lanes per BRAM group. Each lane holds one channel's data. Default: 16.
signed_bytes	If True, data is treated as signed INT8 (-128 to 127). Should always be True for this model.
depth_image_bram	BRAM depth in 128-bit words for image buffers. 2048 words = 32768 bytes.
depth_kernel_bram	BRAM depth in 128-bit words for kernel buffers. 256 words = 4096 bytes.

> **NOTE:** *The BRAM addresses (0xC0000000 etc.) are fixed by the hardware design. Do not change them unless you have a different bitstream with different address assignments.*

5.1.3 Control Register Map

Inside the constructor, the REG dictionary defines the byte offsets of all control registers in the ControlSig hardware IP. Writing to these addresses controls the accelerator:

```
self.REG = {
    'START'      : 0x00,   # Write 1 to start convolution
    'IMG_KER_SIZE': 0x04,  # Encodes image size, kernel size, stride
    'SLIDE'      : 0x08,   # Number of output positions (H x W)
    'BIAS_CTRL'  : 0x0C,   # 1 = add bias after accumulation
    'BIAS_BASE'  : 0x10,   # Base address of bias value registers
    'STATUS'     : 0x50,   # Read bit 0: 1 = done, 0 = busy
    'RESET'      : 0x54,   # Write 1 then 0 to reset the core
    'RELUEN'     : 0x58,   # 1 = apply ReLU activation
    'MAXPOOLEN'  : 0x5C,   # 1 = run max-pooling instead of conv
    'CHANNELRST' : 0x60,   # Resets accumulator across channels
    'DEPTHENB'   : 0x64,   # 1 = depth mode (multi-channel input)
    'LASTCHANNEL': 0x68,   # 1 = this is the final input channel
}
```

5.1.4 DMA Buffer Allocation

The constructor also pre-allocates DMA buffers in DDR RAM. These are special memory regions that the CDMA hardware can access directly to move data to and from the BRAMs:

```
# For each BRAM group, 4 buffers are allocated:
self.img_src_bufs   # Source buffer: image data going TO the FPGA
self.img_dst_bufs   # Destination buffer: results coming BACK from FPGA
self.ker_src_bufs   # Source buffer: kernel weights going TO the FPGA
self.ker_dst_bufs   # Destination buffer: kernel readback (for debug)
```

NOTE: *Each buffer is allocated as dtype='V16' (a 128-bit void type), because each BRAM word is 128 bits wide — holding 16 bytes of data in parallel.*

5.1.5 Loading the Overlay: load_overlay()

This method actually programs the FPGA and maps all IP blocks to Python objects. Call this once before doing anything else:

```
driver.load_overlay()
```

Internally, it does three things:

1. Calls Overlay(bitfile) which downloads the bitstream to the FPGA fabric.
2. Maps CDMA 0 (image transfers) and CDMA 1 (kernel transfers) to MMIO objects.
3. Maps the ControlSig IP to an MMIO object (stored as self.axi_ip) for register access.

5.1.6 Resetting and Enabling the CDMAs

After loading the overlay, the two CDMA engines must be reset and then enabled before any data transfers can occur:

```
driver.reset_cdma(driver.io_fmap_cdmas)    # Reset image CDMA
driver.enable_cdma(driver.io_fmap_cdmas)   # Enable image CDMA

driver.reset_cdma(driver.kernel_cdmas)     # Reset kernel CDMA
driver.enable_cdma(driver.kernel_cdmas)    # Enable kernel CDMA
```

IMPORTANT: *Always call reset_cdma then enable_cdma in this order after loading the overlay. Skipping this step will cause CDMA transfers to hang or produce incorrect results.*

5.1.7 Configuring the Hardware: configure()

Before each convolution or pooling operation, you must configure the hardware by writing the layer's parameters into the control registers. This is done via the configure() method:

```
driver.configure(
    image_size = 28,      # Width/height of the input feature map
    kernel_size = 5,      # Width/height of the convolution kernel
    stride = 1,           # Convolution stride
    slide_h = 24,         # Number of horizontal output positions
    slide_v = 24,         # Number of vertical output positions
    bias_en = 1,          # 1 = add bias to result
    bias_vals = [...],    # List of 16 bias values (one per output channel)
    depth_en = 1,         # 1 = accumulate across input channels
    relu_en = 1,          # 1 = apply ReLU after convolution
    maxpool_en = 0,       # 1 = run max-pool instead of convolution
)
```

The slide_h and slide_v values determine how many output pixels are produced. For a valid convolution with no padding:

```
slide_h = image_size - kernel_size + 1
slide_v = image_size - kernel_size + 1
# Example: 28 - 5 + 1 = 24 output positions in each direction
```

5.1.8 Running the Hardware: run()

After configuring and loading data, call run() to start the hardware computation:

```
driver.run(last_channel=0)   # last_channel=1 on the final input channel
```

This method writes 1 to the START register, then polls the STATUS register in a loop until bit 0 becomes 1 (indicating completion). It then writes 0 to START to clear it.

NOTE: *The last_channel argument is important for multi-channel convolutions(standard convolution). Set it to 1 only on the final channel iteration to tell the hardware to write the final accumulated result to the output BRAM.*

5.1.9 Transferring Data: write and read methods

The firmware provides dedicated methods to move data between DDR RAM and the FPGA BRAMs using the CDMA engines.

Writing image data to the FPGA:

```
driver.write_fmap_to_cdma(
    packed_words_128_image,    # List of numpy arrays: shape (depth, 16)
    fmap_depth,                # Number of 128-bit words to transfer
    lanes,                     # Number of active channels (1-16)
)
```

Reading output feature maps back from the FPGA:

```
fmap_out = driver.read_fmap_from_cdma(
    fmap_depth   # Number of 128-bit words to read
)
# Returns: list of numpy arrays, shape (fmap_depth, 16)
# Access a specific lane: fmap_out[0][:, channel_index]
```

Writing kernel weights to the FPGA:

```
driver.write_kernel_to_cdma(
    packed_words_128_kernel,   # List of numpy arrays: shape (depth, 16)
    kernel_depth,              # Number of 128-bit words to transfer
    lanes,                     # Number of active kernel output channels
)
```

NOTE: *Data is always packed as 128-bit words (16 bytes per word). Each byte corresponds to one channel's value at that spatial position. Unused channels are zero-padded.*

5.2 LeNetInt8Inference Class

This class sits on top of DriverConfig and implements the full LeNet-5 inference pipeline. It knows about the neural network structure and orchestrates the sequence of hardware and software operations needed to classify an input image.

5.2.1 Constructor: __init__()

The constructor loads the quantised model weights from the .npz file:

```
model = LeNetInt8Inference(
    driver  = driver,                                  # DriverConfig object
    npz_path = 'LeNet_data/lenet_int8_fpga_working_95.npz'
)
```

After construction, the following attributes are available (though you rarely need to access them directly):

Attribute	Contents
model.conv1_w / conv1_b	Conv Layer 1 weights (shape: 6,1,3,3) and biases (shape: 6)
model.conv2_w / conv2_b	Conv Layer 2 weights (shape: 16,6,3,3) and biases (shape: 16)
model.fc1_w / fc1_b	FC Layer 1 weights (shape: 120,400) and biases (shape: 120)
model.fc2_w / fc2_b	FC Layer 2 weights (shape: 84,120) and biases (shape: 84)
model.fc3_w / fc3_b	FC Layer 3 weights (shape: 10,84) and biases (shape: 10)
model.SCALE_FACTOR	Quantisation scale multiplier for INT32 -> INT8 rescaling
model.ROUND_FACTOR	Rounding offset applied before the bit-shift
model.SHIFT	Number of right-shift bits in quantisation

5.2.2 Image Loading and Preprocessing: load_image_int8()

Before an image can be sent to the FPGA, it must be preprocessed into the same INT8 format the model was trained with. This is handled automatically inside predict_digit(), but the method is also accessible directly:

```
x_int8 = model.load_image_int8('digit2.png')
# Returns: numpy array of shape (1, 28, 28), dtype int8
```

The preprocessing pipeline is:

1. Open the image and convert to grayscale (single channel).
2. Resize to 28×28 pixels if not already that size.
3. Normalise pixel values from [0, 255] to [0.0, 1.0] by dividing by 255.
4. Apply MNIST standardisation: subtract mean 0.1307, divide by std 0.3081.
5. Quantise to INT8: multiply by 127 and round, then clip to [-128, 127].

5.2.3 Quantisation Rescaling: hw_quant()

After each convolution, the hardware produces INT32 accumulations (because multiplying two INT8 values and summing produces larger numbers). These must be rescaled back to INT8 before the next layer. This is done by hw_quant():

```
# Internally called after each hardware convolution:
acc = acc * SCALE_FACTOR   # Scale to target range
acc = acc + ROUND_FACTOR   # Add rounding offset
acc = acc >> SHIFT         # Right-shift (equivalent to divide by 2^SHIFT)
acc = clip(acc, -128, 127) # Clamp to INT8 range
```

NOTE: *The SCALE_FACTOR, ROUND_FACTOR, and SHIFT values are calibrated during model quantisation and are stored in the .npz file. You do not need to compute or modify them. This is the function that correctly replicates the quantization process in the hardware*

accelerator and is used for requantization of results of the fully connected layers, else the requantization is done by the accelerator itself.

5.2.4 Fully-Connected Layer: fc_int8()

The three fully-connected layers run entirely in software using NumPy. The computation is a standard matrix multiply followed by bias addition:

```
# result = x @ W^T + b
# All operands are cast to int32 to prevent overflow
x = model.fc_int8(x, model.fc1_w, model.fc1_b)
```

5.2.5 The Full Inference Pipeline: predict_digit()

This is the main method you will call. It accepts a path to an image and returns the predicted digit. It also prints timing information for each layer.

```
model.predict_digit(
    img_path   = 'digit2.png',
    show_image = True,          # If True, displays the image in the notebook
)
```

The method goes through the following stages in order:

Stage 1 — Convolution Layer 1 (Hardware)

- The 28×28 INT8 image is flattened to 784 values.
- The image pixels are replicated across all 6 output channels (since conv1 has only 1 input channel, every kernel sees the same input).
- The 3×3 kernels for all 6 output filters are packed into a (9, 6) matrix.
- Both are written to the FPGA BRAMs via CDMA.
- Hardware is configured with image_size=28, kernel_size=3, stride=1, relu_en=1, depth_en=1.
- Hardware runs. Output is a (26×26, 6) feature map read back over CDMA.

Stage 2 — Max-Pool Layer 1 (Hardware)

- The Conv1 output is written back to the FPGA.
- Hardware is configured with kernel_size=2, stride=2, maxpool_en=1.
- A 2×2 max-pool reduces each channel from 26×26 to 13×13.
- Output is read back: shape (169, 6).

Stage 3 — Convolution Layer 2 (Hardware, channel-by-channel)

- Conv2 has 6 input channels and 16 output filters. The hardware processes one input channel at a time.
- A loop runs 6 times (for each input channel i = 0 to 5):

```
for i in range(6):           # 6 input channels from max-pool 1
    # Extract channel i from the feature map
    # Write channel i's image data to FPGA
    # Write corresponding slice of conv2 kernels to FPGA
    # Run hardware with last_channel=0 for i<5 (accumulate)
    # Run hardware with last_channel=1 for i=5 (write final result)
```

- On the last iteration (i=5), the hardware finalises the accumulated result, applies ReLU, and writes to the output BRAM.
- Bias is added only on the final channel iteration (bias_en=1 on i=5 only).

Stage 4 — Max-Pool Layer 2 (Hardware)

- Conv2 output (11×11 × 16 channels) is pooled down to 5×5 × 16.

Stages 5–7 — Fully-Connected Layers FC1, FC2, FC3 (Software)

- The 5×5×16 = 400 values are flattened and reshaped to a 1D vector.
- FC1: 400 → 120 neurons, ReLU activation.
- FC2: 120 → 84 neurons, ReLU activation.
- FC3: 84 → 10 neurons (one score per digit class 0–9).
- The digit with the highest score is returned as the prediction: np.argmax(x).

5.3 Main Execution Script

The main block at the bottom of the firmware file shows you exactly how to put everything together. Here it is reproduced with explanatory comments:

```
# Step 1: Define FPGA memory addresses
FMAP_IN_BRAM_ADDR  = [0xC0000000]   # Input image BRAM
KERNEL_BRAM_ADDR   = [0xC0000000]   # Kernel weight BRAM (same as input here)
FMAP_OUT_BRAM_ADDR = [0xC2000000]   # Output feature map BRAM

# Step 2: Create the hardware driver
driver = DriverConfig(
    bitfile            = 'LeNet_cdma_try4.bit',
    FMAP_IN_BRAM_ADDR  = FMAP_IN_BRAM_ADDR,
    KERNEL_BRAM_ADDR   = KERNEL_BRAM_ADDR,
    FMAP_OUT_BRAM_ADDR = FMAP_OUT_BRAM_ADDR,
    lanes_per_bram     = 16,
    signed_bytes       = True,
    depth_image_bram   = 2048,
```

```
    depth_kernel_bram  = 256,
)

# Step 3: Load overlay and initialise CDMAs
driver.load_overlay()
driver.reset_cdma(driver.io_fmap_cdmas)
driver.enable_cdma(driver.io_fmap_cdmas)
driver.reset_cdma(driver.kernel_cdmas)
driver.enable_cdma(driver.kernel_cdmas)

# Step 4: Load model weights
model = LeNetInt8Inference(
    driver,
    'LeNet_data/lenet_int8_fpga_working_95.npz'
)

# Step 5: Run inference
model.predict_digit('digit2.png', show_image=True)
```

NOTE: *In a Jupyter Notebook, each of the five steps above can go in its own cell. Steps 1–3 only need to be run once per session. Steps 4–5 can be re-run for different images without reloading the overlay.*

6. API Reference

This section provides a concise reference for every public method in both classes. Use this as a quick lookup when writing your own code.

6.1 DriverConfig API

Method Signature	Returns	Description
`load_overlay()`	`None`	Programs FPGA with the bitstream; maps all IPs to Python MMIO objects. Must be called before any other method.
`reset_cdma(cdmas)`	`None`	Soft-resets the given list of CDMA objects. Call after load_overlay().
`enable_cdma(cdmas)`	`None`	Enables the given CDMA objects for transfers. Call after reset_cdma().
`reset()`	`None`	Resets the convolution/pooling hardware core via the RESET register. Call before each layer.
`channel_reset()`	`None`	Resets the internal channel accumulator. Call before multi-channel convolutions (Conv1, Conv2 first pass).
`configure(image_size, kernel_size, stride, slide_h, slide_v, bias_en, bias_vals, depth_en, relu_en, maxpool_en)`	`None`	Writes all layer parameters to the control registers. Must be called before run().
`run(last_channel=0)`	`None`	Starts computation. Blocks until the STATUS register signals completion.

`write_fmap_to_cdma(data, depth, lanes)`	`None`	Transfers image/feature map data from DDR to input BRAM via DMA.
`read_fmap_from_cdma(depth)`	`List[ndarray]`	Reads output feature map from output BRAM back to DDR. Returns list of (depth, 16) int8 arrays.
`write_kernel_to_cdma(data, depth, lanes)`	`None`	Transfers kernel weight data from DDR to kernel BRAM via DMA.
`read_kernel_from_cdma(depth)`	`List[ndarray]`	Reads kernel data back from BRAM (primarily for debugging).

configure() — Parameter Details

Parameter	Type	Valid values	Meaning
`image_size`	int	`Any positive int`	Width (= height) of the square input feature map
`kernel_size`	int	`2, 3, 5, ...`	Width (= height) of the square convolution kernel
`stride`	int	`1 or 2`	Step size for sliding the kernel across the image
`slide_h`	int	`image_size - kernel_size + 1`	Number of horizontal output positions
`slide_v`	int	`image_size - kernel_size + 1`	Number of vertical output positions
`bias_en`	int	`0 or 1`	1 = add bias values after accumulation; 0 = skip bias
`bias_vals`	list[int]	`16 integers`	Bias values for up to 16 output channels. Pad with zeros if fewer channels.
`depth_en`	int	`0 or 1`	0 = accumulate results across multiple input channels (required for Conv2 and max-pool)
`relu_en`	int	`0 or 1`	1 = apply ReLU (zero out negatives) after accumulation
`maxpool_en`	int	`0 or 1`	1 = run max-pooling operation; 0 = run convolution

6.2 LeNetInt8Inference API

Method Signature	Returns	Description
`__init__(driver, npz_path)`	`None`	Loads model weights and quantisation parameters from the .npz file.
`load_image_int8(path)`	`ndarray (1,28,28) int8`	Opens an image, resizes it to 28x28, and quantises it to INT8 with MNIST normalisation.
`relu_int8(x)`	`ndarray int8`	Applies ReLU activation: clips all negative values to 0.
`fc_int8(x, w, b)`	`ndarray int32`	Computes a fully-connected layer: result = x @ w.T + b (in INT32 precision).
`hw_quant(acc)`	`ndarray int8`	Rescales INT32 accumulations to INT8 using ScaleFactor, RoundFactor, and Shift from the .npz file.
`predict_digit(img_path, show_image=True)`	`None (prints result)`	Runs the full LeNet inference pipeline on the given image and prints the predicted digit and layer timing.

7. Running Your First Inference

This section gives you a step-by-step checklist for getting your first digit prediction running in a Jupyter Notebook.

Pre-flight Checklist

- PYNQ board is powered on and accessible in your browser.
- LeNet_cdma_try4.bit and LeNet_cdma_try4.hwh are in the working directory.
- LeNet_data/lenet_int8_fpga_working_95.npz is in the LeNet_data/ subfolder.
- A test image (28×28 or larger grayscale PNG) is uploaded to the working directory.
- The firmware Python file is imported or pasted into the notebook.

Cell-by-Cell Guide

Run each of the following notebook cells in order:

Cell 1 — Imports

```
from pynq import Overlay, MMIO, allocate
import numpy as np
import time
from PIL import Image
import matplotlib.pyplot as plt
# (If firmware is in a separate .py file:)
# from lenet_driver import DriverConfig, LeNetInt8Inference
```

Cell 2 — Create the Driver

```
FMAP_IN_BRAM_ADDR  = [0xC0000000]
KERNEL_BRAM_ADDR   = [0xC0000000]
FMAP_OUT_BRAM_ADDR = [0xC2000000]

driver = DriverConfig(
    bitfile            = 'LeNet_cdma_try4.bit',
    FMAP_IN_BRAM_ADDR  = FMAP_IN_BRAM_ADDR,
    KERNEL_BRAM_ADDR   = KERNEL_BRAM_ADDR,
    FMAP_OUT_BRAM_ADDR = FMAP_OUT_BRAM_ADDR,
    lanes_per_bram     = 16,
    signed_bytes       = True,
    depth_image_bram   = 2048,
    depth_kernel_bram  = 256,
)
```

Cell 3 — Load Overlay and Initialise CDMAs

```
driver.load_overlay()
```

```
driver.reset_cdma(driver.io_fmap_cdmas)
driver.enable_cdma(driver.io_fmap_cdmas)
driver.reset_cdma(driver.kernel_cdmas)
driver.enable_cdma(driver.kernel_cdmas)

print('Overlay loaded. FPGA ready.')
```

IMPORTANT: *Cell 3 programs the FPGA. It will take a few seconds. Wait for the cell to finish before proceeding.*

Cell 4 — Load the Model

```
model = LeNetInt8Inference(
    driver,
    'LeNet_data/lenet_int8_fpga_working_95.npz'
)
print('Model weights loaded.')
```

Cell 5 — Run Inference

```
model.predict_digit('digit2.png', show_image=True)
```

Expected Output

If everything is working correctly, you should see:

- The input image displayed above the cell output.
- The following printed output (with your own timing values):

```
predicted digit:  2

Time taken for convolution layer 1  :  3.42 ms
Time taken for maxpool layer 1      :  1.21 ms
Time taken for convolution layer 2  :  12.87 ms
Time taken for maxpool layer 2      :  1.18 ms
Total Inference time (including memory overhead) :  45.30 ms
```

NOTE: *The total inference time includes all CDMA transfer overhead (DDR to BRAM and back) in addition to pure computation time. The hardware computation itself is typically only a few milliseconds.*

8. Understanding the Output

Predicted Digit

The prediction is made by looking at 10 output scores (one per digit 0–9) produced by the final fully-connected layer (FC3). The digit with the highest score is selected using np.argmax().

If you want to see the raw scores instead of just the winning digit, you can modify predict_digit() to also return x before calling argmax — these raw scores give you a confidence indication.

Timing Breakdown

The firmware prints five timing values. Here is what each one means:

Timing Label	What it measures
Time taken for convolution layer 1	Hardware execution time from START=1 to STATUS=1 for Conv1.
Time taken for maxpool layer 1	Hardware execution time for the 2x2 max-pooling after Conv1.
Time taken for convolution layer 2	Sum of hardware execution times across all 6 channel iterations of Conv2.
Time taken for maxpool layer 2	Hardware execution time for the 2x2 max-pooling after Conv2.
Total Inference time (including memory overhead)	Wall-clock time from image preprocessing to final argmax — includes all CDMA transfers and Python overhead.

> **NOTE:** *The total inference time will always be larger than the sum of the individual layer times because it includes all CDMA data transfer time, Python processing overhead, and MMIO polling latency.*

9. Troubleshooting

Problem	Likely Cause and Solution
FileNotFoundError on .bit or .hwh file	The bitstream files are not in the same directory as the notebook. Upload both LeNet_cdma_try4.bit and LeNet_cdma_try4.hwh to the Jupyter working directory.
FileNotFoundError on .npz file	The model weights file is missing or in the wrong location. Ensure the file is at LeNet_data/lenet_int8_fpga_working_95.npz relative to your notebook.
CDMA transfer hangs (cell runs forever)	CDMA was not properly reset and enabled. Restart the kernel, re-run all cells from the beginning, and ensure reset_cdma() is called before enable_cdma().
Overlay() throws an error about .hwh file	The .hwh hardware handoff file is missing. Both .bit and .hwh must be in the same directory with identical base filenames.
Prediction is always the same digit regardless of input	The image may not be loading correctly. Print the output of load_image_int8() and check that the pixel values vary across images. Also ensure the image is genuine grayscale (not RGB).

ImportError for pynq or PIL	You are not running in the PYNQ environment. These notebooks must be executed directly on the PYNQ board, not on a regular PC.
Wrong prediction despite correct setup	The input image may be too low contrast, too small, or not centred. The model was trained on MNIST-style digits (white digit on black background, centred). Try inverting the image if your digit is black on white.
AttributeError: DriverConfig has no attribute 'axi_ip'	load_overlay() was not called. Always call driver.load_overlay() before configure(), run(), or any CDMA methods.

General Debugging Tips

- If the board seems unresponsive, restart the Jupyter kernel and re-run from Cell 1. The FPGA state is cleared when the kernel restarts.
- Always run the cells in order. Running Cell 5 before Cell 3 will fail because the overlay has not been loaded yet.
- If you get unexpected results from a layer, temporarily add a print statement after read_fmap_from_cdma() to inspect the raw output array before passing it to the next stage.
- The commented-out debug code at the bottom of the firmware file (marked with #) shows how to test CDMA transfers in isolation by writing a known image and reading it back to confirm data integrity.

NOTE: *When in doubt, the safest recovery is: Kernel > Restart & Run All in the Jupyter menu. This reloads the overlay and reinitialises all hardware state from scratch.*